# Conference Paper Title*

*Note: Sub-titles are not captured for https://ieeexplore.ieee.org and should not be used

Josè Dalla Torre
*Machine Learning and Perception Lab*
*University of Udine*
Udine, Italy
dallatorre.josezito@spes.uniud.it

Christian Micheloni
*Machine Learning and Perception Lab*
*University of Udine*
Udine, Italy
christian.micheloni@uniud.it

Gian Luca Foresti
*Machine Learning and Perception Lab*
*University of Udine*
Udine, Italy
gianluca.foresti@uniud.it

***Abstract*—This and the IEEEtran.cls file define the components of your paper [title, text, heads, etc.]. *CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract.**

***Index Terms*—component, formatting, style, styling, insert.**

## I. Introduction

TODO: write Introduction
In order to understand the salient actions of a broadcast soccer game, SoccerNet introduces the task of action spotting, which consists in finding all the actions occurring in the videos. Beyond soccer understanding, this task addresses the more general problem of retrieving moments with a specific semantic meaning in long untrimmed videos. Introduce temporal action spotting in broadcast soccer. Explain that the model must identify both the action category and its timestamp in an untrimmed video, and that ball events can be brief and closely spaced. Introduce temporal 3D convolutions as a way to learn features across neighboring frames. Explain their local temporal processing and why you want to investigate self-attention as an alternative. Avoid claiming that convolution is inherently inadequate; your experiments should determine whether the alternative helps. Explain that your work investigates whether changing the temporal feature encoder, while preserving the rest of the pipeline, changes accuracy and efficiency on this particular task. The gap should be supported by related literature, not merely by the fact that you implemented a Transformer. List only contributions you actually make. For example:

- An implementation of a Transformer-based temporal encoder in an existing ball action spotting pipeline.
- A controlled comparison against a 3D CNN temporal encoder.
- An analysis of accuracy, temporal context, and computational cost.

## II. Related Work

### A. First solution with paper

Here we talk about the solution with the paper.
Our method, Boosted Model Ensembling (BME), is based on the end-to-end baseline model, E2E-Spot, as presented in [10]? citation.
We generate several variants of the E2E-Spot model to create a candidate model set and propose a strategy for selecting appropriate model members from this set while assigning appropriate weights to each selected model. More details can be found in [35]? citation.
The resulting ensemble model takes into account uncertainties in event length, optimal network architectures, and optimizers, making it more robust than the baseline model. Our approach has the potential to handle various video event analysis tasks.

### B. Second solution I based my solution on

Solution Key points:
Efficient model architecture for extracting information from video data Multi-stage training (transfer learning, fine-tuning with long sequences) Fast video loading for training (GPU based, no need for preprocessing with extracting images) Model The model architecture employed in this solution utilizes a slow fusion approach, incorporating 2D convolutions in the early stages and 3D convolutions in the later stages. This architectural choice made one of the most significant contributions to the overall metric outcome. It effectively improved the mAP@1 metric on both the test and challenge sets, resulting in an approximate increase of 15% (from 65% to 80%) compared to an approach utilizing 2D CNN early fusion.
model
The model consumes sequences of grayscale frames. Neighboring frames are stacking as channels for input to the 2D convolutional encoder. For example, if fifteen frames are stacked in sets of three, the outcome would be five input tensors, each consisting of three channels. The shared 2D encoder independently processes these input tensors, producing visual features. The following 3D encoder processes visual features, producing temporal features. Concated temporal features pass through global pooling to compress the spatial dimensions. Then, a linear classifier predicts the presence of actions in the middle frame of the original input sequence.
I choose the following model hyperparameters as a result of the experiments:
Stack threes from 15 grayscale 1280x736 frames skipping every second frame from the original 25 FPS video (equivalent to 15 neighboring frames at 12.5 FPS, about 1.16 seconds

window) EfficientNetV2 B0 as 2D encoder 4 inverted residual 3D blocks as 3D encoder (ported from 2D EfficientNet version) GeM as global pooling Multilabel classification with positive labels applied in a 0.6 seconds window (15 frames at 25 FPS) around the action timestamp from annotation You can find more details in the model implementation and experiment configs.

Training I made several stages of training to obtain 86.47% mAP@1 on the challenge set (87.03% on the test):

Basic training (config). The 2D encoder is initialized with ImageNet-pre-trained weights; other parts start from scratch. Training on Action Spotting Challenge dataset (config). Same weights initialization as in the first stage. Transfer learning (config). 2D and 3D encoders initialized with weights obtained in the second stage. Out-of-fold predictions from the first stage were used for data sampling (more details later). Fine-tuning with long sequences (config). 2D and 3D encoders initialized with weights obtained in the third stage. 2D encoder weights are frozen. Basic training In this challenge, I employed 7-fold cross-validation to tune the training pipeline more precisely. Each labeled game from the dataset is an individual fold.

In short, the resulting training pipeline hyperparameters are:

Learning rate warmup for the first 6 epochs from 0 to 3e-4, cosine annealing last 30 epochs to 3e-6 Batch size 4, one training epoch comprises 6000 samples Optimizer AdamW with weight decay 0.01 Focal loss with gamma 1.2 Model EMA with decay 0.999 Initial weights for 2D encoder ImageNet pretrained Architecture hyperparameters listed above in the model part Worth writing about sampling techniques during training, which significantly impacts its results. For basic training, a simple but well-working sampling algorithm was used. For each training sample, randomly take a video index from a uniform distribution. Then randomly choose a frame index with an equal chance to sample frames near to actions and remaining frames. Frame near to action if allocated in a 0.36 seconds window (9 frames at 25 FPS) around the action. I tried different ratios, but an equal chance to show empty and event frame worked best. I will introduce a more advanced sampling scheme in the section on transfer learning.

I applied the usual augmentations like horizontal flip, rotation, random resized crop, brightness, motion blur, etc. Of the interesting, there are only two temporal augmentations:

Simulating camera movement (change translation, scale, and angle over time) Randomly shake the frames in sequences (applied 40% chance to change the frame index on +- 1) The models from this training have 79.06% on CV (cross-validation) and 84.26% mAP@1 on the test set (the metric on test split was calculated by the out-of-fold predictions for two folds which include test games). I did not evaluate these models for the challenge set.

Training on Action Spotting Challenge dataset I built a similar pipeline for videos and classes from Action Spotting Challenge to get good initial weights for the next experiment.

Briefly, here are the changes from the previous:

377 games in training, 18 in validation split 15 classes (all cards-related class was merged because the model consumes grayscale frames) 4 warmup epochs and 20 training epochs, one epoch is 36000 samples Weight the frame sampling by the effective number of class samples Transfer learning This training uses the results of the previous two. The second one gives excellent initial weights for 2D and 3D encoders. It provides a significant boost ( 2% mAP@1 on test and CV). That is understandable because the same models were trained on many games with similar input frames and some similar actions.

Basic training gives out-of-fold predictions that I use for sampling in the following way:

sampling

Take the element-wise maximum between the sampling distribution (introduced above) and predictions, then normalize again to equal probability sums between empty and action frames. The intuition is that there are some hard negative examples in the dataset. Due to many negative samples, such hard examples are rarely sampled during training. We can make something like hard negative mining/sampling with the technique.

Other minor changes compared to basic training:

7 warmup epochs and 35 training epochs Focal loss with gamma 1.2 and alpha 0.4 Models achieve 81.04% on CV, 86.51% on the test, and 86.35% mAP@1 on the challenge set.

Fine-tuning with long sequences Before, I trained all models on relatively short clips (15 frames in 12.5 FPS, 1.16 seconds) to fits the VRAM of a single GPU and to decrease training time. Models obtained good 2D and 3D encoder weights in the previous experiment. So I tried to freeze 2D weights and fine-tune 3D weights on long sequences to provide more context.

Changes compared to transfer learning experiment:

33 frames in 12.5 FPS (2.6 seconds) LR warmup first 2 epochs from 0 to 1e-3, cosine annealing last 7 epochs to 5e-05 SGD with Nesterov momentum 0.9 Models scored 80.49% on CV, 87.04% on the test, and 86.47% mAP@1 on the challenge set. The score is lower on cross-validation, but it's my best submission on the test and the challenge set.

Prediction and postprocessing Models predict each possible sequence of frames from the videos. Additionally, I make test time augmentation with the horizontal flip. On the challenge set, I used the arithmetic mean of predictions from all fold models.

Postprocessing is very simple. I just used a combination of Gaussian filter and peak detection from SciPy with the following parameters: standard deviation for Gaussian kernel 3.0, peak detection minimal height 0.2, and minimal distance between neighboring peaks 15 frames.

Training and prediction accelerations I optimized the training pipeline to iterate experiments faster and to test more hypotheses.

Custom multiprocessing video loader with simultaneous use VideoProcessingFramework (GPU decoding) and OpenCV (CPU decoding) workers to optimize hardware utilization FP16 with Automatic Mixed Precision torch.compile using TorchDynamo backend Augmentation on the GPU with kornia

These accelerations allow running epoch (train + val) of basic training in 7 minutes and 10 seconds on a single RTX 3090 Ti. It is impressive because one epoch is 6000 training and approximately 2600 validation examples, each of which is 15 frames in 1280x736 resolution. Also, using source videos without the preprocessing with extracting images allows using any video frame during training and saves disk space.

I applied caching strategy to speed up inference time using the architecture structure. If one saves the last visual features, it is enough to predict with the 2D encoder only one stack of frames when receiving a new one. The 2D encoder is the most time expensive part of the model. Predicting 3D features takes a short time. So this strategy dramatically boosts prediction speed by several times.

My solution is very inspired by the top solutions of the DFL - Bundesliga Data Shootout competition:

Team Hydrogen (link) K_mat (link) Camaro (link) ohkawa3 (link)

askldjkljekle

### C. Transformer-Based Video Modeling

Introduce self-attention, positional information, and the use of Transformer encoders to model temporal sequences. Include work on temporal action spotting and video modeling that is technically relevant to your architecture. For instance, Transformer-based temporal reasoning has already been explored in soccer action spotting, so your paper should distinguish your particular replacement and evaluation from prior work rather than claim to introduce Transformers to the task.

### D. C. Transformer-Based Temporal Modeling

To investigate the use of self-attention for temporal feature modeling, we replace the original 3D convolutional encoder with a Transformer encoder while retaining the shared 2D visual feature extractor. The input video is divided into groups of three consecutive grayscale frames, and each group is processed independently by the 2D encoder. The resulting feature maps are projected to a common feature dimension of 192 channels.

Unlike the original architecture, which performs temporal modeling on spatial feature maps, the proposed approach first applies adaptive average pooling to each feature map, producing a sequence of five 192-dimensional feature vectors for a 15-frame input. Learned positional embeddings are added to the sequence to provide information about the temporal ordering of the feature vectors.

The resulting sequence is processed by a Transformer encoder consisting of two layers with eight attention heads and an embedding dimension of 192. Self-attention allows each temporal token to incorporate information from the other tokens in the input sequence. The encoded representations are then aggregated by temporal average pooling to obtain a single 192-dimensional video representation.

Finally, a linear classification layer maps this representation to the output action classes. This architecture replaces the original 3D convolutional temporal encoder and its subsequent feature projection and concatenation with a Transformer-based temporal encoder and average-pooling operation. The effect of this architectural change is evaluated using the same action spotting task and evaluation metric as the baseline.

### E. Convolutional Temporal Modeling

### F. Transformer-Based Video Modeling

## III. Proposed Method

### A. Task Definition

Define the input video sequence, the action classes, the timestamp-based labels, and the prediction target. Explain whether the model predicts the action at the center frame or predicts a temporal sequence of action probabilities.

Your draft describes labels expanded around the annotated timestamp. Make that explicit, including the window size, because it affects what the model is learning.

This novel task of ball action spotting consists in localizing the exact time when two types of actions related to the soccer ball occur: pass and drive. These actions are anchored with a single timestamp corresponding to the exact time the ball leaves a player's foot for a pass event and when a player touches the ball to control it for a drive event.
Ball action spotting involves identifying both the timing and type of ball-related actions across 12 classes in our dataset. Each action is marked by a single timestamp. In contrast to the SoccerNet Action Spotting Challenge, our videos feature a higher density of actions.

### B. Soccer Action Spotting

ball action spotting, focusing on retrieving all timestamps related to the soccer ball change of state,

### C. Baseline Architecture

**Research question is supposed to be about replacing the temporal 3D CNN with a Transformer** Describe the existing pipeline with a diagram and a concise explanation: 1. Sample frames from the video. 2. Stack neighboring grayscale frames into groups of three. 3. Extract visual features using the shared 2D encoder. 4. Process the feature sequence using the 3D temporal encoder. 5. Pool the features and predict action probabilities. The draft already contains most of this information. Add tensor dimensions at each major stage, where possible, so that the reader can reproduce the architecture.

### D. Baseline Architecture

This should be the main contribution. Specify: - Which features enter the Transformer: raw frame embeddings, spatially pooled 2D features, or intermediate CNN feature maps. - The projection from visual feature dimension to Transformer embedding dimension. - The positional encoding method. -

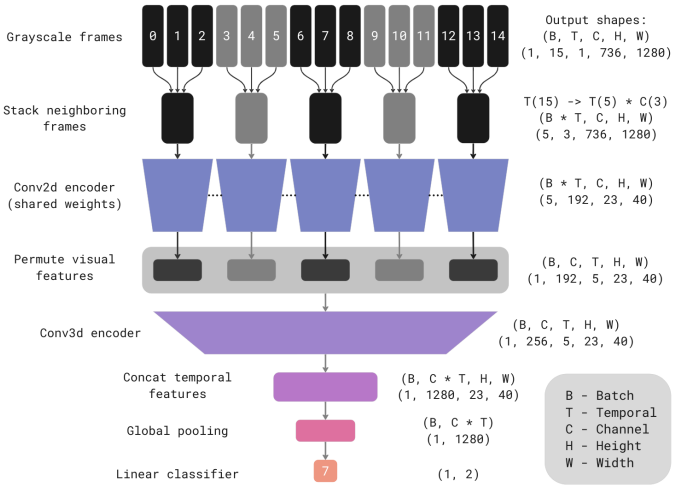



Fig. 1. Model second solution

Number of Transformer layers and attention heads. - Feed-forward dimension, normalization, dropout, and activation. - How the temporal representation is pooled or selected for classification. Explain which parts remain unchanged from the baseline and which are replaced.

### *E. Training and Inference*

Describe the common training setup, loss, label expansion, augmentation, and prediction postprocessing. If the Transformer requires different optimization settings or a different sequence length, report those differences clearly.

A key methodological choice is whether you train the Transformer from scratch or initialize it from pretrained weights. That choice affects how you interpret the results, particularly with limited labeled data.

### *F. Temporal Feature Representation*

### *G. Transformer Encoder*

### *H. Prediction and Postprocessing*

## IV. Experimental Setup

The goal here is to make the experiment reproducible and the comparison defensible. Subsection What it should contain Dataset Number of matches, classes, annotations, and data split Implementation Framework, hardware, input resolution, model settings Evaluation metrics mAP@1 and any efficiency metrics Experimental configurations Baseline, Transformer variant, and ablations Training protocol Initialization, optimizer, schedule, seeds, and validation procedure

Keep the primary comparison as controlled as possible: use the same data splits, input frames, 2D feature extractor, label construction, postprocessing, and evaluation implementation for both models.

If you change several things at once — for example, the temporal encoder, the input length, and the loss — then a performance difference cannot be attributed confidently to the Transformer.

### *A. Implementation Details*

### *B. Evaluation Metrics*

### *C. Experimental Configurations*

## V. Results and Discussion

Your current proposed subsections are sensible, but each should answer a specific question.

A. Main Comparison

Compare the CNN and Transformer using the same evaluation protocol. Report mAP@1, absolute difference in percentage points, and, if feasible, variation across random seeds or folds.

B. Effect of Temporal Context

Test whether performance changes when the input sequence becomes longer. To isolate the effect of the encoder, use comparable temporal windows for both architectures.

C. Ablation Study

Test one design choice at a time, such as embedding dimension, number of Transformer layers, or positional encoding. Choose only a few ablations that help explain the main result.

D. Error Analysis

Investigate false positives, missed actions, closely spaced events, and differences between action classes. Use examples or per-class metrics where possible.

I'd also report inference latency and peak GPU memory if efficiency is part of the research question. Parameter count alone is not a sufficient measure of computational efficiency.

One distinction matters for your current draft: the leaderboard table is useful context, but it is not a substitute for a controlled baseline comparison. Your model must be evaluated on the same split and with the same protocol before you can attribute a change to the temporal encoder.

We evaluate the proposed Transformer-based model on the 2024 ball-action spotting challenge using the official evaluation protocol. We report the primary challenge metric and describe the evaluation split and inference procedure.

### *A. Dataset*

The dataset is provided by Footovision and is composed of 7 games with a total of 11,041 annotated timestamps, as well as 2 extra segregated games for the challenge set.

The dataset is [1]

The data consists of 7 videos of English Football League games, each broadcast in 720p quality. We don't provide extracted features to encourage the development of end-to-end methods. The challenge set is composed of 2 separate games. Additionally, participants are encouraged to leverage the 500 unannotated games from the Action Spotting challenge as a resource for this task.

The dataset available is relatively small compared to the action spotting task, encouraging participants to explore semi-supervised, selfsupervised, or transfer learning methods using the 500 unlabeled broadcast games.

A significant enhancement this year is the inclusion of 12 classes of ball actions, requiring more precise classification.

TABLE I

BALL ACTION SPOTTING LEADERBOARD. THE MAIN METRIC FOR THE LEADERBOARD AND BEST PERFORMANCES ARE IN BOLD. TEAM NAMES WITH A SUPERSCRIPT HAVE PROVIDED A SUMMARY THAT MAY BE FOUND IN APPENDIX **??** OR IN SECTION **??** FOR THE WINNING TEAM.

| Participants | mAP @1 | mAP @2 | mAP @3 | mAP @4 | mAP @5 | Average-mAP tight |
|---|---|---|---|---|---|---|
| **Ruslan Baikulov**$^{B1}$ | **86.47** | **87.98** | **88.28** | **88.18** | **87.95** | **87.91** |
| FDL@ZLab$^{B2}$ | 83.39 | 85.19 | 85.81 | 86.00 | 86.19 | 85.45 |
| BASIK$^{B3}$ | 82.06 | 83.39 | 83.86 | 84.04 | 83.91 | 83.57 |
| FC Pixel Nets$^{B4}$ | 81.89 | 83.22 | 83.97 | 83.85 | 84.02 | 83.50 |
| play | 79.74 | 82.58 | 84.06 | 84.49 | 84.34 | 83.29 |
| Baseline [**?**]* | 62.72 | 69.24 | 72.57 | 74.29 | 74.80 | 71.21 |

TABLE II

BALL ACTION SPOTTING LEADERBOARD. THE MAIN METRIC FOR THE LEADERBOARD AND BEST PERFORMANCES ARE IN BOLD.

Consequently, simple categorizations like "pass" are insufficient as specific pass types must be clearly identified.
We provide annotation for all two types of soccer ball actions: Pass, Drive. In 2024, we extend it to 12 different classes of soccer ball actions: Pass, Drive, Header, High Pass, Out, Cross, Throw In, Shot, Ball Player Block, Player Successful Tackle, Free Kick, Goal

### B. Evaluation

Unlike the SoccerNet Action Spotting Challenge, that uses the tight average-mAP ([1-5] seconds tolerance), this challenge focuses on accurately spotting the ball events with the mAP@1 (1 second tolerance).

### C. Comparison with the Baseline

We use the 3D CNN-based architecture from the 2023 challenge as the reference for our design. Since the proposed model was trained on the 2024 data, results from the two challenge years are not a controlled comparison of temporal architectures. Differences in dataset composition, training data, and evaluation conditions must be considered when interpreting the results.

Here we need to compare to other results.
After the table

### D. Effect of Temporal Context

### E. Ablation Study

### F. Error Analysis

We examine the model's predictions to identify common false positives, missed actions, and temporal localization errors. Where ground-truth labels are available, we also analyze performance by action class.

### G. Limitation

The current comparison does not isolate the effect of replacing 3D convolutions with self-attention. The proposed model also changes spatial and temporal feature aggregation. A controlled comparison using both architectures trained and evaluated under the same conditions would be needed to attribute performance differences more specifically to the temporal modeling approach

## VI. CONCLUSION AND FUTURE WORK

### A. LaTeX-Specific Advice

"Fig. 2", even at the beginning of a sentence.

TABLE III
TABLE TYPE STYLES

| Table Head | Table Column Head | | |
|---|---|---|---|
| | ***Table column subhead*** | ***Subhead*** | ***Subhead*** |
| copy | More table copy[a] | | |

[a]Sample of a Table footnote.

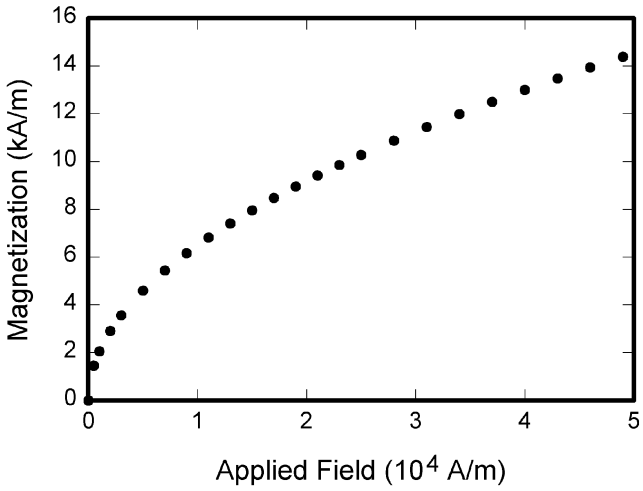



Fig. 2. Example of a figure caption.


## ACKNOWLEDGMENT

Acknowledgment


## REFERENCES

References blah blah

## REFERENCES


[1] A. Deliège, A. Cioppa, S. Giancola, M. J. Seikavandi, J. V. Dueholm, K. Nasrollahi, B. Ghanem, T. B. Moeslund, and M. V. Droogenbroeck, "Soccernet-v2 : A dataset and benchmarks for holistic understanding of broadcast soccer videos," in *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, June 2021.



IEEE conference templates contain guidance text for composing and formatting conference papers. Please ensure that all template text is removed from your conference paper prior to submission to the conference. Failure to remove the template text from your paper may result in your paper not being published.